🐳 Docker — Layering & Port Binding

> Topic-only study notes • Taught in 3 stages + Interview Corner • With diagrams Examples tailored to a JavaScript / Next.js / MongoDB stack.

Learner: Akshay • **Date:** 6 August 2026

Part A — Image Layering

Docker Image Layering

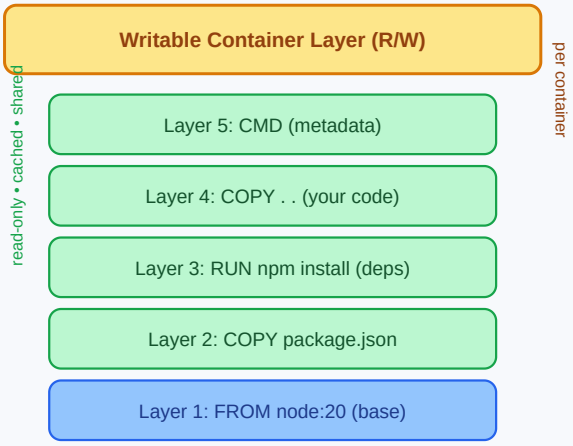
Each Dockerfile instruction adds a read-only layer. The container adds a writable layer on top.

Dockerfile

```
FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm","start"]
```

→

Image = stacked layers



Key idea:
The blue base layer (node:20) is stored ONCE and shared by every image built on it. Change only your code → only Layer 4 & 5 rebuild; Layers 1-3 come

🟢 Beginner

A Docker image is built in **layers stacked like a cake** 🎂 — each Dockerfile instruction creates one layer. The bottom is the base (`node:20`); running the image adds one **writable layer** on top (the only part a container can change).

Analogy: transparent projector sheets stacked together — each adds something, and looking through all of them shows the complete picture. Docker flattens the layers into one filesystem for your app.

🟡 Working Developer

Each layer is **read-only and cached**. On rebuild, Docker reuses cached layers and only rebuilds changed layers *and everything above them*. So **instruction order matters**:

```
# GOOD — deps cached separately from code
FROM node:20
WORKDIR /app
COPY package*.json ./     # changes only when deps change
RUN npm install           # expensive, but cached
COPY . .                  # volatile code sits ABOVE npm install
CMD ["npm","start"]
```

```
# BAD — any code change re-runs npm install
FROM node:20
WORKDIR /app
COPY . .                  # busts cache on every code edit...
RUN npm install           # ...so install re-runs every build
```

Inspect layers:

```
docker history node:20    # each layer + size
docker image inspect node:20
```

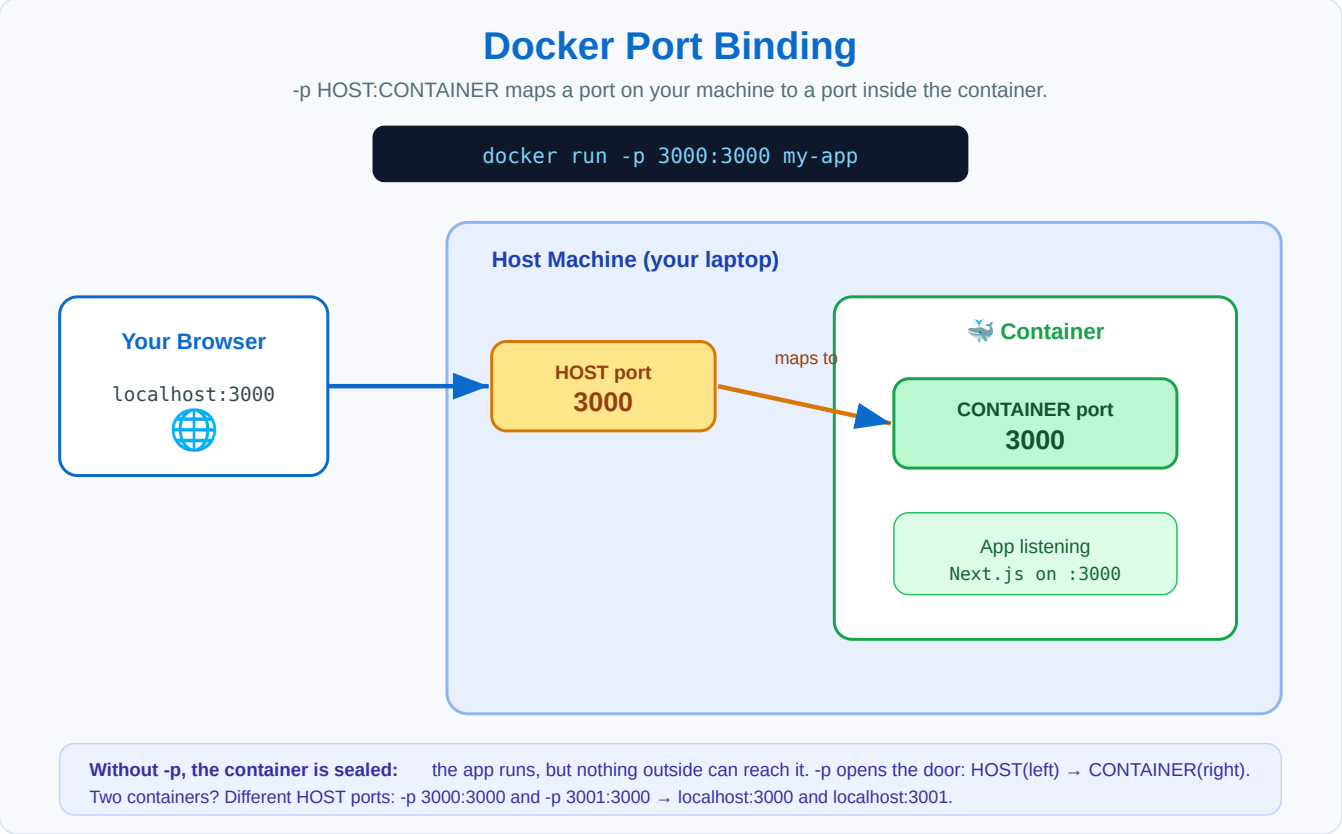
🔴 Expert

- **Content-addressed & immutable:** each layer has a SHA-256 hash; identical layers are stored **once** and shared across images.
- **Cache invalidation is a chain:** a layer is reused only if it *and all layers below it* are unchanged. One change rebuilds everything above it.
- **Union filesystem (OverlayFS):** layers merge into one `/`; reads served top-down.
- **Copy-on-write:** writing a file from a read-only layer copies it up to the writable layer first; base untouched.
- **Best practices:** order least-changed → most-changed; combine `RUN` steps with `&&`; use `.dockerignore`; use **multi-stage builds** for small images.

🎯 Interview Corner

- **What are image layers?** Read-only cached filesystem layers (one per instruction), merged via a union FS into one image.
- **Why does instruction order matter?** Layer caching — a changed layer invalidates all above it; put stable steps before volatile ones.
- **How are layers shared?** Content-hashed; identical layers stored once, reused across images.
- **Copy-on-write?** Writing to a read-only file copies it up to the writable layer; base stays untouched.
- ⚠️ **Trap:** `COPY . .` before `RUN npm install` re-runs installs on every code change.

Part B — Port Binding



🟢 Beginner

A container is **sealed** by default — the app runs inside but your browser can't reach it (like a shop with its front door locked). **Port binding opens a door**: the `-p` flag connects a port on your **host** to a port **inside the container**.

Flow: browser → `localhost:3000` → **host port 3000** → **container port 3000** → your app listening inside.

🟡 Working Developer

Format is always `-p HOST:CONTAINER` (left = your machine, right = inside container):

```
docker run -p 3000:3000 my-app   # localhost:3000 → container 3000
docker run -p 8080:3000 my-app   # localhost:8080 → container 3000
```

The **right side must match** the port the app listens on inside; the **left side is your choice**.

MERN example:

```
docker run -d -p 3000:3000 my-next-app   # frontend → localhost:3000
docker run -d -p 5000:5000 my-api        # backend  → localhost:5000
docker run -d -p 27017:27017 mongo       # database → localhost:27017
```

Two of the same app → unique **host** ports:

```
docker run -d -p 3000:3000 my-app   # localhost:3000
docker run -d -p 3001:3000 my-app   # localhost:3001
```

🔴 Expert

- `EXPOSE` **vs** `-p` **:** `EXPOSE 3000` is **documentation only** — publishes nothing. Only `-p` (or `-P`) actually binds a port.
- `-P` **(capital):** auto-publishes all `EXPOSE`d ports to random high host ports; check with `docker port <container>`.
- **Bind to an interface:** `-p 127.0.0.1:3000:3000` = localhost only; plain `-p 3000:3000` binds `0.0.0.0`.
- **Protocol:** default TCP; UDP via `-p 53:53/udp`.
- **Container-to-container:** doesn't use `-p` — same Docker **network**, reach by name (`mongodb://mongo:27017`).
- **Common failures:** app bound to `127.0.0.1` inside (must be `0.0.0.0`); wrong side of mapping; host port already allocated.

🎯 Interview Corner

- **What does** `-p 8080:80` **mean?** Host 8080 → container 80.
- **Which side is host/container?** `-p HOST:CONTAINER` (left host, right container).
- `EXPOSE` **vs** `-p` **?** `EXPOSE` documents only; `-p` / `-P` actually publishes.
- **Why listen on** `0.0.0.0` **inside?** Binding to `127.0.0.1` inside makes the app unreachable via the mapping.
- **How do two containers talk?** Shared Docker network + names, not `-p`.
- ⚠️ **Traps:** thinking `EXPOSE` publishes; swapping host/container sides; host ports must be unique.